



Automatic Meeting Minutes & Action Item Extraction

An end-to-end NLP system that turns meeting audio into auditable summaries, action items, owners, deadlines, and review decisions.

NLP FINAL PROJECT

PRODUCTION DESIGN

AGENTIC AI

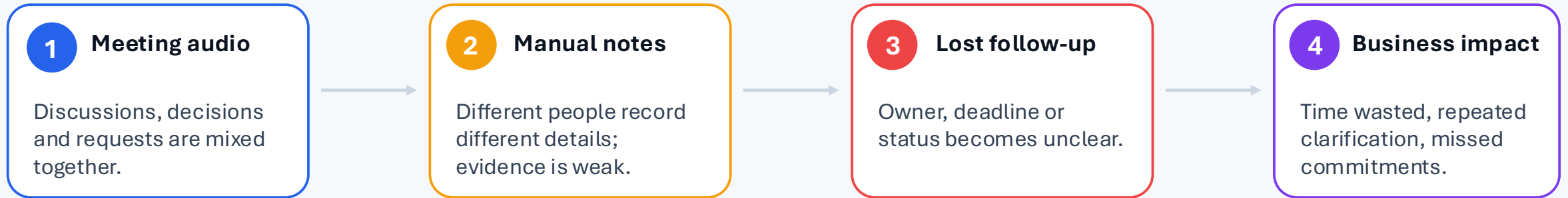
Course Applications of Natural Language Processing in Industry

Lecturer TS. Nguyễn Hồng Bửu Long | TS. Lương An Vinh

Student Nguyễn Văn Minh Thiện — 22127398

Business problem: meetings create untracked commitments

Meeting decisions often stay inside audio, personal notes, or chat messages.
The business risk is not transcription; it is losing accountability after the meeting.



Target users

Project managers, technical leads, product owners and team members who need traceable post-meeting tasks.

Why NLP is required

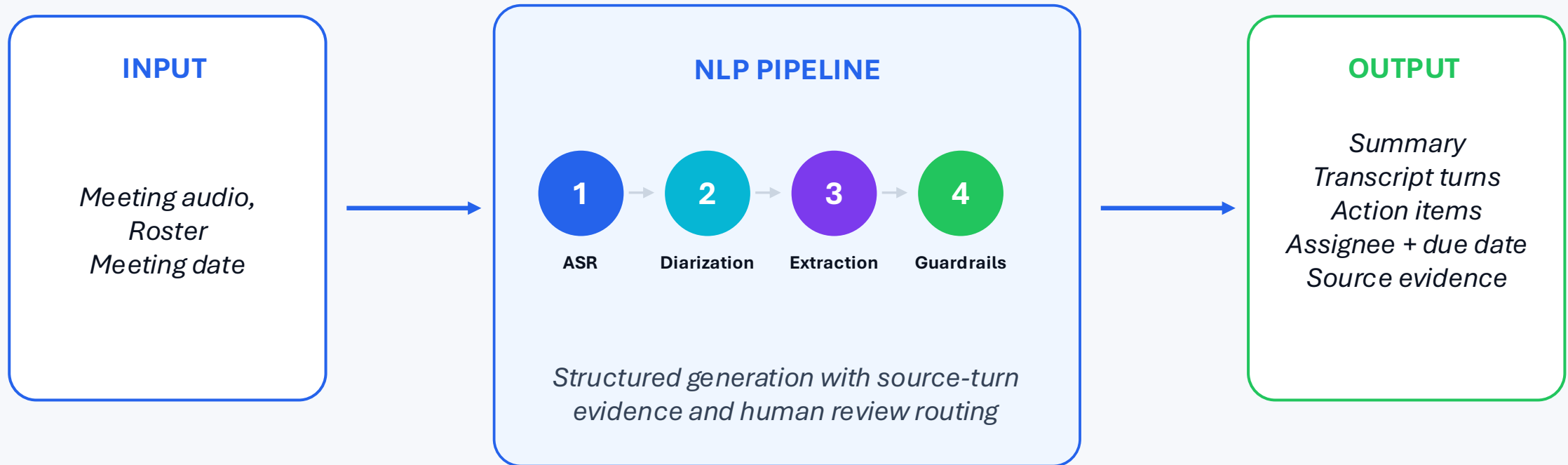
The system must infer commitments, assignees, relative deadlines and source evidence from natural conversation.

Success criteria

Reduce manual minutes; improve task traceability; maintain precision, F1, schema stability and operational latency.

Product objective: turn speech into accountable work items

The system is designed as a deployable product: *audio ingestion, asynchronous processing, structured output, review UI, persistence and monitoring.*



Core claim

The value is the structured, auditable task object — not only a transcript.

Production claim

Async jobs, database state and metrics make long-running NLP processing operationally feasible.

Risk claim

Ambiguous or low-confidence outputs are routed to review instead of being silently trusted.

NLP task formulation: from dialogue to verified JSON

The system decomposes one messy meeting into smaller NLP and data-engineering tasks, then validates the result before persistence.

Example transcript turn

“Bob, can you send the report by Friday?”

Required interpretation

- detect commitment
- map alias “Bob” to roster
- normalize relative due date
- retain source turn for audit

Structured action item

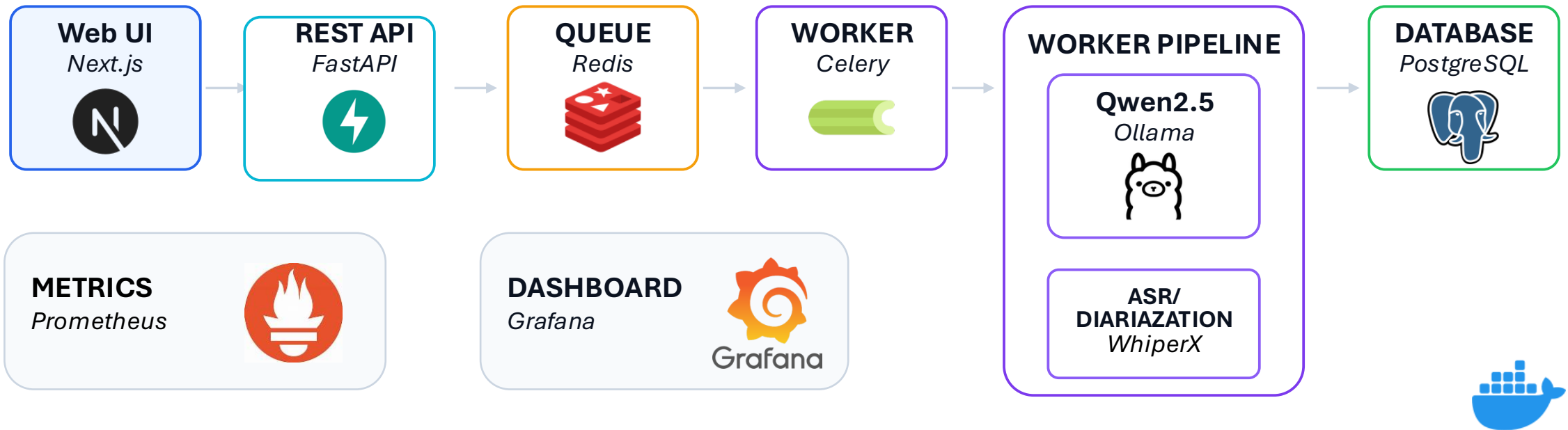
```
{  
  "description": "send the report",  
  "assignee": "Robert Kim",  
  "due_date": "<meeting Friday>",  
  "priority": "normal",  
  "source_turn_ids": [...]  
}
```



Overview architecture

Docker Compose separates the interactive product surface from heavy asynchronous NLP stages and observability.

Docker Compose



Why async?

ASR, diarization and local LLM inference exceed normal HTTP request latency; jobs need queueing and persistent state.

Why local LLM?

Ollama/Qwen2.5-3B enables reproducible demo and reduces dependency on external APIs.

Why monitoring?

Production review needs stage latency, failures, task counts and guardrail signals.

Data management: synthetic benchmark + auditable schema

The report uses a reproducible synthetic benchmark while treating runtime audio/transcripts as sensitive data that must not be committed to Git.

205

meeting samples

2,036

transcript turns

1,620 / 416

train / test turns

60.01%

positive turn rate

EN

benchmark language

Data model

Meeting

Transcript Turns

Participants
& Workers

Action Items

Feedback
Corrections

Artifacts & Metrics

Split strategy

Meeting-level 80/20 split prevents transcript turns from the same meeting leaking across train/test.

Limitations

Synthetic benchmark is reproducible but does not fully cover accents, noisy audio, multilingual meetings or real privacy issues.

Feedback data

User corrections become future evaluation/training data after PII scrubbing, validation and deduplication.

Model design: LLM extraction with a real lightweight baseline

The production path extracts structured tasks; the baseline is intentionally smaller and solves action-turn detection as a reproducible comparison point.

Main runtime pipeline

- **WhisperX** ASR with timestamps
- **Diarization** speaker labels
- **Qwen2.5-3B** summary + JSON tasks
- **Guardrails** schema, PII, evidence checks

Traditional baseline

- **Input** transcript turn text
- **Features** TF-IDF unigram/bigram
- **Classifier** Logistic Regression
- **Task** action-turn detection only

Evaluation results: high precision, weaker recall and assignee resolution

The strongest result is stable JSON and low hallucination flags;
the main improvement target is assignee mapping and missed action items.

LLM precision

0.8604

*Most predicted tasks
match reference items.*

LLM recall

0.6665

*Some reference action
items are missed.*

LLM F1

0.6886

*Reported by benchmark
script, not recomputed
from rounded P/R.*

Assignee accuracy

0.5232

*Hard due to aliases, self-
assignment and indirect
ownership.*

Schema
failures

0.0%

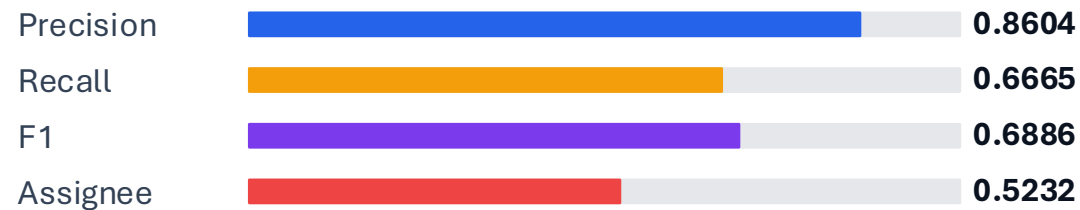
*JSON schema was
stable on the
benchmark.*

Hallucination
flags

0.0%

*No task outside
transcript by current
heuristic.*

LLM extraction metrics



Baseline: action-turn detection



Latency: average 26.97s; P95 120.2s → handled as asynchronous post-meeting processing.

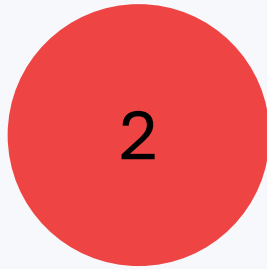
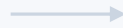
Error analysis: where the system still needs engineering

The reported metrics point to precise but incomplete extraction.
The next iteration should focus on recall and ownership resolution.



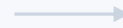
Indirect commitments

“Maybe someone can...”
not an explicit task



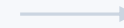
Assignee ambiguity

alias, pronoun,
self-assignment



Relative deadlines

“next Friday”
needs meeting date



Noisy transcript

ASR error or
speaker overlap

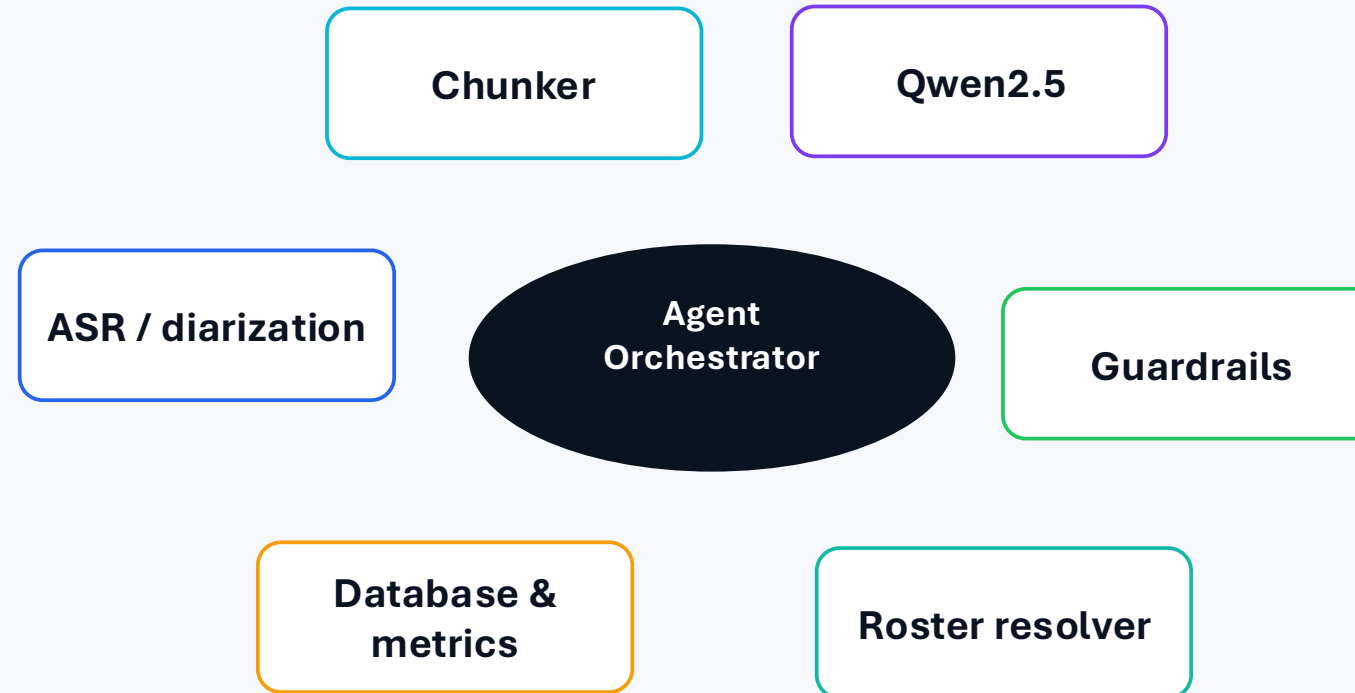
Engineering response

*keep source-turn evidence
improve roster/alias resolver
add stricter review routing
expand benchmark with real/noisy/multilingual meetings*

Agentic AI: decisions are made from intermediate signals

The agent is not a chatbot wrapper.

It orchestrates tools, validates intermediate outputs and decides whether results can be stored or must be reviewed.



Decision rule example

"Bob" uniquely matches roster alias → save Robert Kim. Ambiguous alias or low confidence → send to human review.

Deployment design: asynchronous API, versioned models, review UI

The deployable interface is a REST/API + web demo over a background pipeline.
Users do not wait for LLM inference inside a single blocking request.



Repository readiness

src/	tests/
data/	pyproject.toml
models/	README
configs/	

Operational constraints

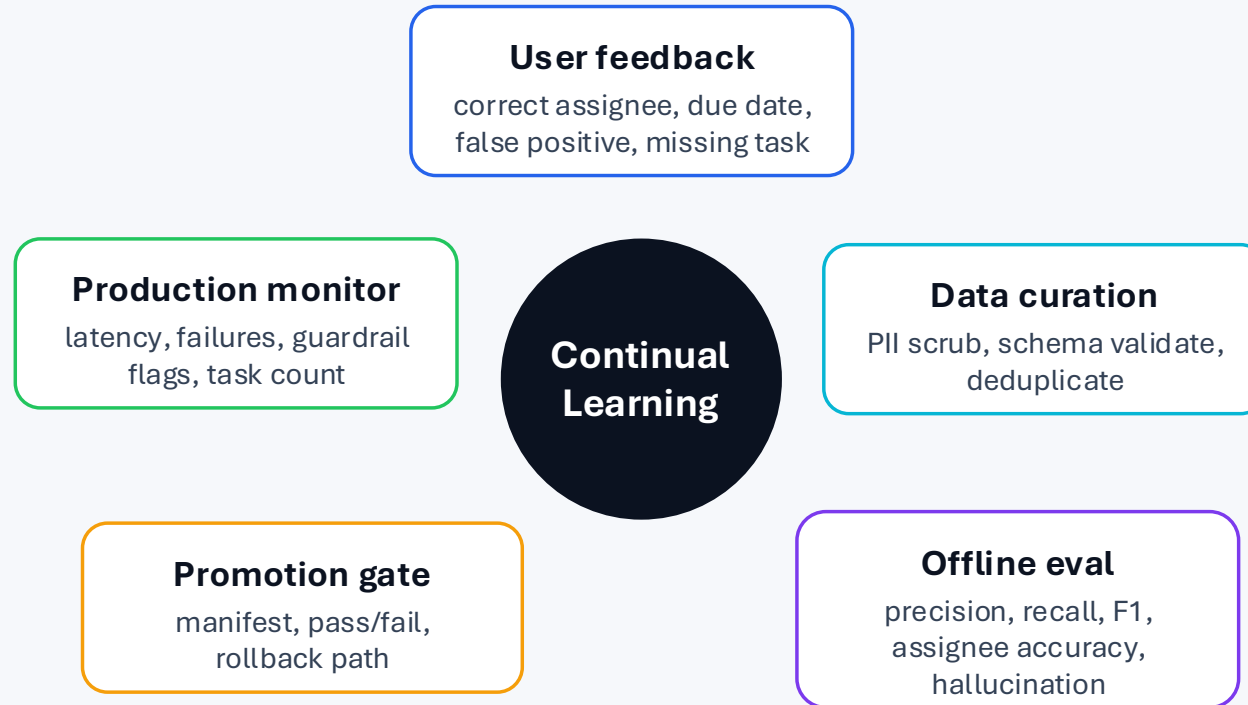
Async jobs for latency
Redis/Celery for queueing
PostgreSQL for durable job state.

Model governance

Model version stored
in meeting metadata.

Monitoring and continual learning loop

New data should improve the system only after validation, PII scrubbing, benchmark evaluation and controlled promotion.



Drift risks

New domains, language mix, microphone quality and roster changes.

Quality gates

Candidate models must beat fixed benchmarks and preserve guardrail stability.

Key takeaways and next iteration

The project satisfies the assignment as an end-to-end NLP system, but the honest engineering story is that production quality depends on review, monitoring and better real-world data.

1. End-to-end system

Audio → ASR/diarization → structured extraction → guardrails → database → web review → monitoring.

2. Evaluation is explicit

LLM extraction metrics and TF-IDF action-turn baseline are separated to avoid invalid comparisons.

3. Risk is handled by design

Ambiguity triggers review; outputs retain evidence; PII and misuse risks are documented.

Future work roadmap



Real data eval

consent + anonymized meetings



Audio robustness

noise, overlap, accents



Human-in-loop

approval workflow + audit logs

Final message: the system should be trusted as a decision-support assistant, not as an automatic source of truth.

Resources & references

Project deliverables and academic references used throughout this work.

PROJECT DELIVERABLES

Data

[minhthien/mia-meeting](#)

Source Code

[nguyenvmthien/MIA](#)

Video Demo

(add link)

REFERENCES

[1]

[2]

[3]

[4]

[5]

[6]

[7]

[8]